

UNIT-1

1.Characteristic of IOT

CHARACTERISTICS OF IOT

There are 5 types of characteristics that are:

- 1) Dynamic & Self-Adapting.
- 2) Self-Configuring.
- 3) Interoperable Communication Protocols.
- 4) Unique Identity.
- 5) Integrated into Information Network.

1) Dynamic & Self Adapting: IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user's context or sensed environment.

Eg: the surveillance system is adapting itself based on context and changing conditions.

2) Self Configuring: allowing a large number of devices to work together to provide certain functionality.

3) Inter Operable Communication Protocols: support a number of interoperable communication protocols and can communicate with other devices and also with infrastructure.

4) Unique Identity: Each IoT device has a unique identity and a unique identifier (IP address).

5) Integrated into Information Network: that allow them to communicate and exchange data with other devices and systems.

2.Application of iot

APPLICATIONS OF IOT [OR] DOMAIN SPECIFIC IOTs

The Applications on IoT networks extract and create information from lower- level data by filtering, processing, categorizing, condensing and contextualizing the data.

- 1) Home.
- 2) Cities.
- 3) Environment.
- 4) Energy.
- 5) Retail.
- 6) Logistics.
- 7) Agriculture.
- 8) Industry.
- 9) Health & Lifestyle.

1) Home Automation:

- i. **Smart Lighting:** helps in saving energy by adapting the lighting to the ambient conditions and switching on/off or dimming the light when needed.
- ii. **Smart Appliances:** make the management easier and also provide status information to the users remotely.
- iii. **Intrusion Detection:** use security cameras and sensors (PIR sensors and door sensors) to detect intrusion and raise alerts. Alerts can be in the form of SMS or email sent to the user.
- iv. **Smoke/Gas Detectors:** Smoke detectors are installed in homes and buildings to detect smoke that is typically an early sign of fire. Alerts raised by smoke detectors can be in the form of signals to a fire alarm system. Gas detectors can detect the presence of harmful gases such as CO, LPG etc.,

2) Cities:

- i. **Smart Parking:** make the search for parking space easier and convenient for drivers. Smart parking is powered by IoT systems that detect the no. of empty parking slots and send information over internet to smart application backends.
- ii. **Smart Lighting:** for roads, parks and buildings can help in saving energy.
- iii. **Smart Roads:** Equipped with sensors can provide information on driving condition, travel time estimating and alert in case of poor driving conditions, traffic condition and accidents.
- iv. **Structural Health Monitoring:** uses a network of sensors to monitor the vibration levels in the structures such as bridges and buildings.
- v. **Surveillance:** The video feeds from surveillance cameras can be aggregated in cloud based scalable storage solution.
- vi. **Emergency Response:** IoT systems for fire detection, gas and water leakage detection can help in generating alerts and minimizing their effects on the critical infrastructures.

3) Environment:

- i. **Weather Monitoring:** Systems collect data from a no. of sensors attached and send the data to cloud-based applications and storage back ends. The data collected in cloud can then be analyzed and visualized by cloud-based applications.
- ii. **Air Pollution Monitoring:** System can monitor emission of harmful gases (CO_2 , CO , NO , NO_2 etc.,) by factories and automobiles using gaseous and meteorological sensors. The collected data can be analyzed to make informed decisions on pollutions control approaches.
- iii. **Noise Pollution Monitoring:** Due to growing urban development, noise levels in cities have increased and even become alarmingly high in some cities. IoT based noise pollution monitoring systems use a no. of noise monitoring systems that are deployed at different places in a city. The data on noise levels from the station is collected on servers or in the cloud. The collected data is then aggregated to generate noise maps.

- iv. **Forest Fire Detection:** Forest fire can cause damage to natural resources, property and human life. Early detection of forest fire can help in minimizing damage.
- v. **River Flood Detection:** River floods can cause damage to natural and human resources and human life. Early warnings of floods can be given by monitoring the water level and flow rate. IoT based river flood monitoring system uses a no. of sensor nodes that monitor the water level and flow rate sensors.

4) Energy:

- i. **Smart Grids:** is a data communication network integrated with the electrical grids that collects and analyze data captured in near-real-time about power transmission, distribution and consumption. Smart grid technology provides predictive information and recommendations to utilities, their suppliers, and their customers on how best to manage power. By using IoT based sensing and measurement technologies, the health of equipment and integrity of the grid can be evaluated.
- ii. **Renewable Energy Systems:** IoT based systems integrated with the transformers at the point of interconnection measure the electrical variables and how much power is fed into the grid. For wind energy systems, closed-loop controls can be used to regulate the voltage at point of interconnection which coordinate wind turbine outputs and provides power support.
- iii. **Prognostics:** In systems such as power grids, real-time information is collected using specialized electrical sensors called Phasor Measurement Units (PMUs) at the substations. The information received from PMUs must be monitored in real-time for estimating the state of the system and for predicting failures.

5) Retail:

- i. **Inventory Management:** IoT systems enable remote monitoring of inventory using data collected by RFID readers.
- ii. **Smart Payments:** Solutions such as contact-less payments powered by technologies such as Near Field Communication (NFC) and Bluetooth.

- iii. **Smart Vending Machines:** Sensors in a smart vending machines monitors its operations and send the data to cloud which can be used for predictive maintenance.

6) Logistics:

- i. **Route generation & scheduling:** IoT based system backed by cloud can provide first response to the route generation queries and can be scaled up to serve a large transportation network.
- ii. **Fleet Tracking:** Use GPS to track locations of vehicles in real-time.
- iii. **Shipment Monitoring:** IoT based shipment monitoring systems use sensors such as temp, humidity, to monitor the conditions and send data to cloud, where it can be analyzed to detect food spoilage.
- iv. **Remote Vehicle Diagnostics:** Systems use on-board IoT devices for collecting data on Vehicle operations (speed, RPM etc.,) and status of various vehicle subsystems.

7) Agriculture:

- i. **Smart Irrigation:** to determine moisture amount in soil.
- ii. **Green House Control:** to improve productivity.

8) Industry:

- i. Machine diagnosis and prognosis.
- ii. Indoor Air Quality Monitoring.

9) Health and Lifestyle:

- i. Health & Fitness Monitoring.
- ii. Wearable Electronics.

3. Physical Design of Iot

PHYSICAL DESIGN OF IOT

☞ The "Things" in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and monitoring capabilities.

☞ IoT devices can:

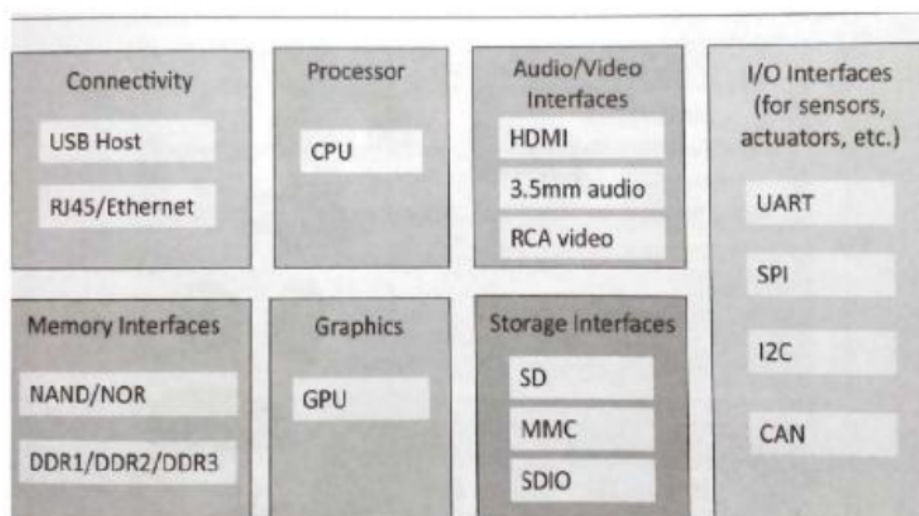
- ↳ Exchange data with other connected devices and applications (directly or indirectly), or
- ↳ Collect data from other devices and process the data locally or
- ↳ Send the data to centralized servers or cloud-based application back-ends for processing the data, or
- ↳ Perform some tasks locally and other tasks within the IoT infrastructure, based on temporal and space constraints.

1) Things in IoT/Generic block diagram of an IoT Device:

☞ The things in IoT refers to IoT devices which have unique identities and perform remote sensing, actuating and monitoring capabilities. IoT devices can exchange data with other connected devices applications. It collects data from other devices and process data either locally or remotely.

☞ An IoT device may consist of several interfaces for connections/communication to other devices, both wired and wireless. These includes:

- i. I/O interfaces for sensors.
- ii. Interfaces for Internet connectivity.
- iii. Memory and storage interfaces.
- iv. Audio/video interfaces.



2) IoT Protocols:

i. Link Layer:

- ☞ 802.3 – Ethernet
- ☞ 802.11 – WiFi
- ☞ 802.16 – WiMax
- ☞ 802.15.4 – LR-WPAN
- ☞ 2G/3G/4G

ii. Network/Internet Layer:

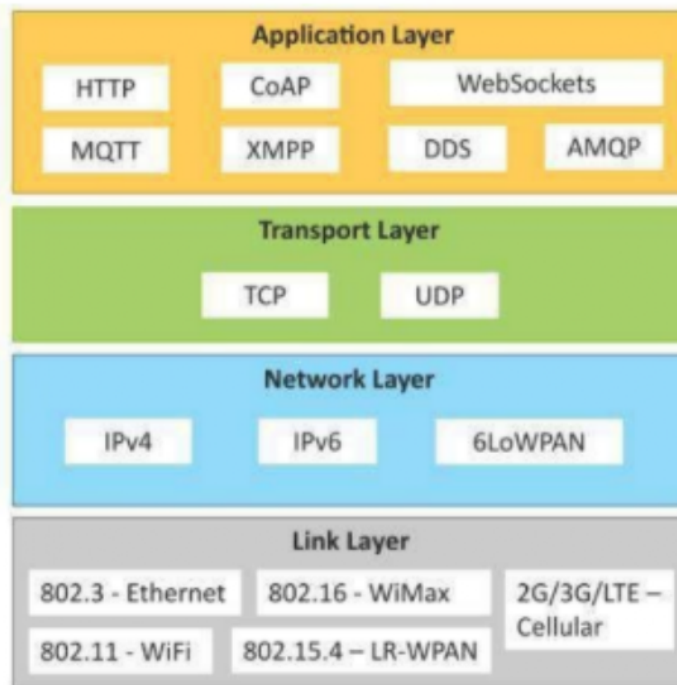
- ☞ IPv4
- ☞ IPv6
- ☞ 6LoWPAN

iii. Transport Layer:

- ☞ TCP
- ☞ UDP

iv. Application Layer:

- ☞ HTTP
- ☞ CoAP
- ☞ WebSocket
- ☞ MQTT
- ☞ XMPP
- ☞ DDS
- ☞ AMQP



i) **Link Layer:** Protocols determine how data is physically sent over the network's physical layer or medium. Local network connect to which host is attached. Hosts on the same link exchange data packets over the link layer using link layer protocols. Link layer determines how packets are coded and signaled by the h/w device over the medium to which the host is attached.

Protocols:

- 🔗 **802.3-Ethernet:** IEEE802.3 is collection of wired Ethernet standards for the link layer. Eg: 802.3 uses co-axial cable; 802.3i uses copper twisted pair connection; 802.3j uses fiber optic connection; 802.3ae uses Ethernet over fiber.
- 🔗 **802.11-WiFi:** IEEE802.11 is a collection of wireless LAN (WLAN) communication standards including extensive description of link layer. Eg: 802.11a operates in 5GHz band, 802.11b and 802.11g operates in 2.4GHz band, 802.11n operates in 2.4/5GHz band, 802.11ac operates in 5GHz band, 802.11ad operates in 60Ghzband.
- 🔗 **802.16 - WiMax:** IEEE802.16 is a collection of wireless broadband standards including exclusive description of link layer. WiMax provide data rates from 1.5 Mb/s to 1Gb/s.

☞ **802.15.4-LR-WPAN:** IEEE802.15.4 is a collection of standards for low rate wireless personal area network (LR-WPAN). Basis for high level communication protocols such as ZigBee. Provides data rate from 40kb/s to 250kb/s.

☞ **2G/3G/4G-Mobile Communication:** Data rates from 9.6kb/s(2G) to up to 100Mb/s(4G).

ii) **Network/Internet Layer:** Responsible for sending IP datagrams from source n/w to destination n/w. Performs the host addressing and packet routing. Datagrams contains source and destination address.

Protocols:

☞ **IPv4:** Internet Protocol version 4 is used to identify the devices on a n/w using a hierarchical addressing scheme. 32 bit address. Allows total of 2^{32} addresses.

☞ **IPv6:** Internet Protocol version 6 uses 128 bit address scheme and allows 2^{128} addresses.

☞ **6LOWPAN:** (IPv6 over Lowpower Wireless Personal Area Network) operates in 2.4 GHz frequency range and data transfer 250 kb/s.

iii) **Transport Layer:** Provides end-to-end message transfer capability independent of the underlying n/w. Set up on connection with ACK as in TCP and without ACK as in UDP. Provides functions such as error control, segmentation, flow control and congestion control.

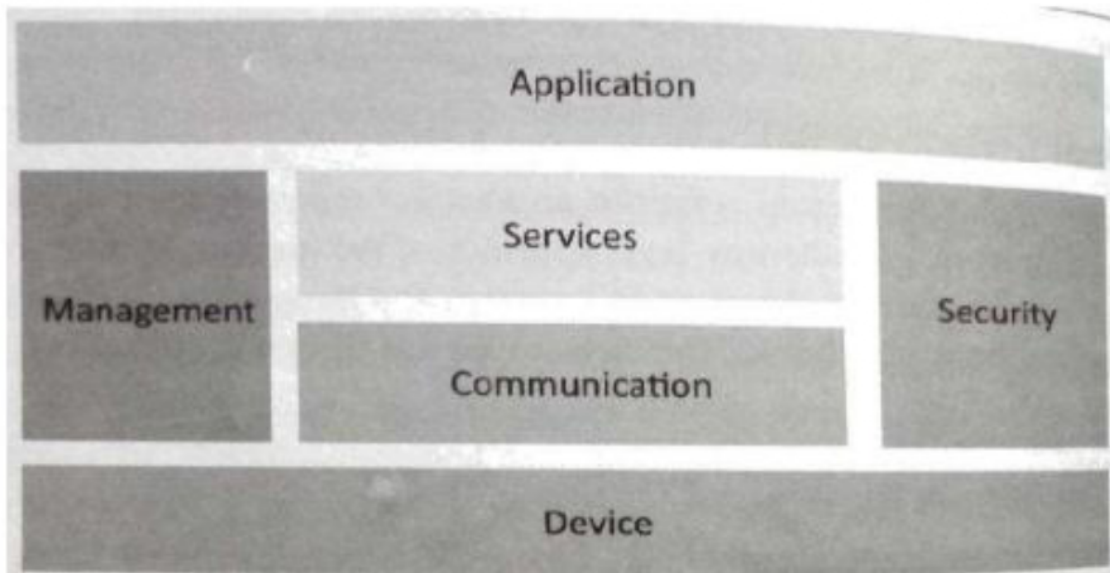
4.Functional Block of Iot

FUNCTIONAL BLOCKS OF IOT [OR] IOT FUNCTIONAL BLOCKS

Provide the system the capabilities for identification, sensing, actuation, communication and management.

- 1) Device.
- 2) Communication.
- 3) Services.
- 4) Management.
- 5) Security.

- 6) Applications.



- 1) **Device:** An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions.
- 2) **Communication:** handles the communication for IoT system.
- 3) **Services:** for device monitoring, device control services, data publishing services and services for device discovery.
- 4) **Management:** Provides various functions to govern the IoT system.
- 5) **Security:** Secures IoT system and priority functions such as authentication, authorization, message and context integrity and data security.
- 6) **Application:** IoT application provide an interface that the users can use to control and monitor various aspects of IoT system.

5.Sensing

SENSING

- 🔗 The main purpose of sensors is to collect data from the surrounding environment. Sensors, or 'things' of the IoT system, form the front end.
- 🔗 These are connected directly or indirectly to IoT networks after signal conversion and processing.

🔗 **Types of IoT sensors:**

- 1) Temperature sensors.
- 2) Humidity sensors.

- 3) Motion sensors.
- 4) Gas sensors.
- 5) Smoke sensors.
- 6) Pressure sensors.
- 7) Image sensors.
- 8) IR sensors.

6.Actuation

What is Actuation?

Actuation is the process of converting a decision or signal (from a processor/controller) into **physical action** using devices called **actuators**.

In the context of IoT, once data is sensed and processed (e.g., temperature exceeds threshold), an actuator performs the required action (e.g., turns on a fan).

How It Works in IoT?

IoT System Flow:

Sensor → Microcontroller → Decision Logic → Actuator

- **Sensor** senses data (e.g., room is too hot).
 - **Controller** (e.g., Arduino/Raspberry Pi) analyzes data.
 - **Actuator** performs action (e.g., turn on cooler/fan).
-

What is an Actuator?

An **Actuator** is a device that:

- Receives control signals from the microcontroller or cloud system.
 - Performs **physical tasks** (movement, turning on/off devices, etc.).
-

Types of Actuators in IoT

Actuator Type	Function	Example Use Case
Electrical	On/off control of circuits	Light bulb switch
Mechanical	Movement or rotation	Servo motor opens a door
Thermal	Generates heat	Electric heater
Hydraulic	Uses fluid pressure for motion	Industrial robotic arms
Pneumatic	Uses compressed air for action	Air-powered valves

Examples in Real IoT Applications

Application	Sensor Input	Actuator Output
Smart Home	Temperature sensor	AC or fan ON/OFF
Smart Agriculture	Soil moisture sensor	Water pump activation
Smart Lighting	Motion sensor	Light ON when motion
Smart Lock	Fingerprint scanner	Servo motor unlocks door

7.Basic Of Internetworking

🌐 Basics of Networking (As per JNTUH Syllabus - FIOT)

📖 What is Networking in IoT?

Networking is the process of **connecting devices (nodes)** together to enable **communication and data exchange**.

In IoT, billions of devices (sensors, smartphones, microcontrollers) need to **communicate** with each other using a **network infrastructure** (wired or wireless).

🔑 Key Networking Concepts

Concept	Description
Node	Any device connected to a network (e.g., sensor, mobile, Raspberry Pi)
Host	A computer/device that can send/receive data
IP Address	A unique identifier for each device in the network
MAC Address	Hardware address assigned to a network interface
Bandwidth	The amount of data that can be transmitted in a given time
Latency	Time taken for data to travel from source to destination

🐾 Types of Networks

Type	Description	Example
PAN	Personal Area Network (few meters)	Bluetooth headset, smartwatch
LAN	Local Area Network (within building)	Home WiFi, office intranet
MAN	Metropolitan Area Network	City-wide network (WiMAX)
WAN	Wide Area Network (large-scale)	Internet, cellular network
LPWAN	Low Power Wide Area Network (IoT-focused)	LoRaWAN, Sigfox

🌐 Network Topologies

Topology	Description	Example Use
Star	Devices connected to a central hub	Home WiFi
Mesh	Each node connected to multiple nodes	ZigBee, WSN
Bus	Single communication line for all nodes	Older LANs
Ring	Each device connects to two others	Token ring

📡 IP Addressing in IoT

- Devices need **unique IP addresses** to communicate.
- IoT uses **IPv6** widely due to the large number of devices (IPv4 has limited addresses).

Example:

- **IPv4:** 192.168.0.1
- **IPv6:** 2001:0db8:85a3:0000:0000:8a2e:0370:7334

🔌 Wired vs Wireless Networks

Wired	Wireless
Ethernet cables	WiFi, ZigBee, LoRa
Reliable, fast	Flexible, low-cost
Costly installation	Easy to deploy

📊 Importance of Networking in IoT

- Ensures **real-time communication** between devices.
- Enables **remote monitoring and control**.
- Supports **cloud connectivity** and data analytics.

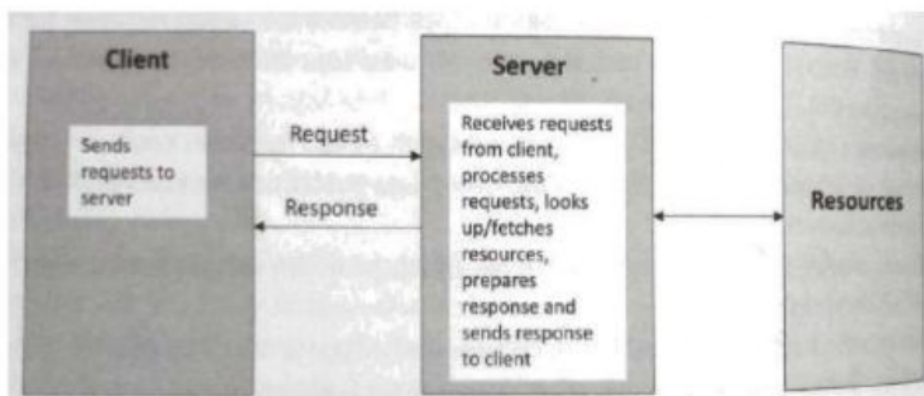
8.Communication Protocol

COMMUNICATION PROTOCOLS

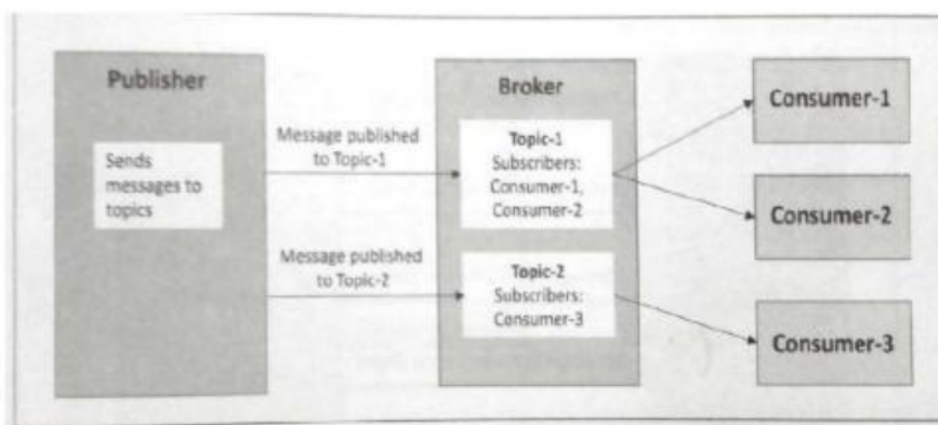
IoT Communication Models:

- 1) Request-Response Model.
- 2) Publish-Subscribe Model.
- 3) Push-Pull Model.
- 4) Exclusive Pair Model.

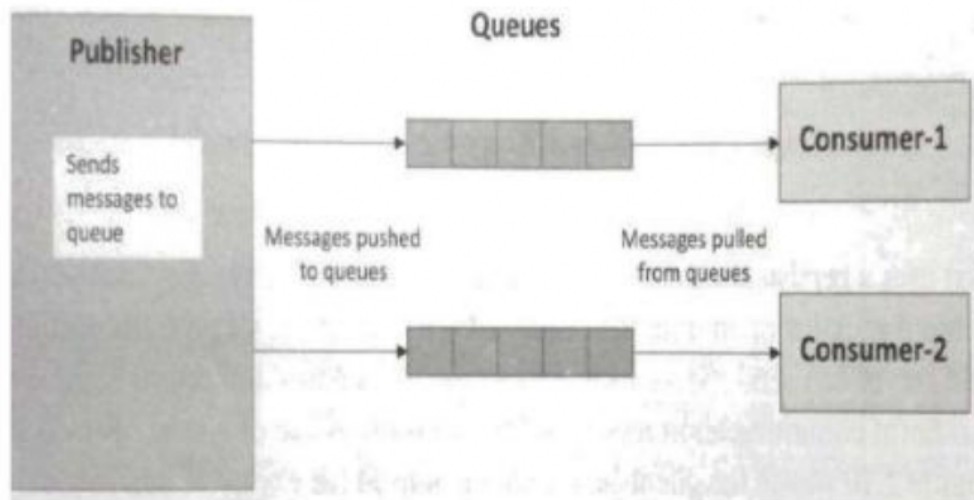
1) Request-Response Model: In which the client sends request to the server and the server replies to requests. Is a stateless communication model and each request-response pair is independent of others.



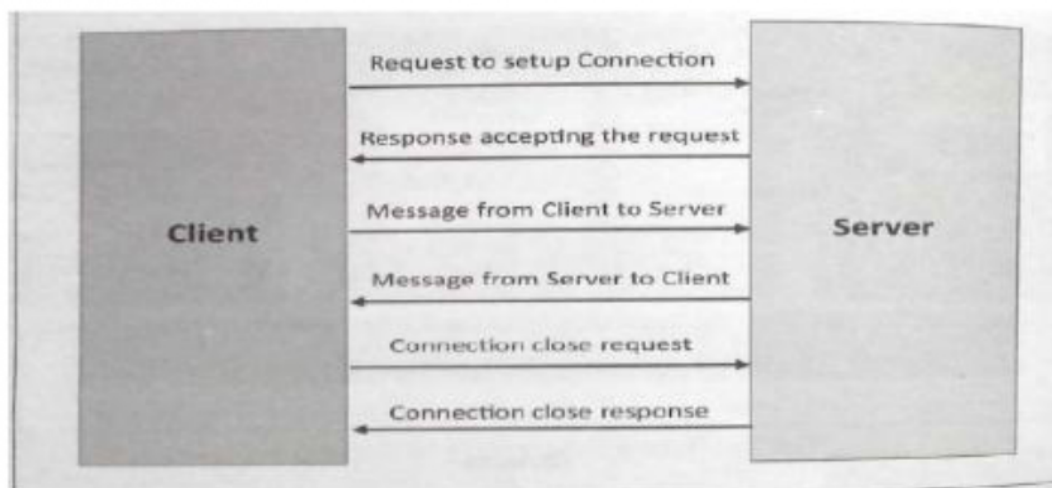
2) Publish-Subscribe Model: Involves publishers, brokers and consumers. Publishers are source of data. Publishers send data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers.



3) Push-Pull Model: in which data producers push data to queues and consumers pull data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the message between the producers and consumers.



4 Exclusive Pair: is bi-directional, fully duplex communication model that uses a persistent connection between the client and server. Once connection is set up it remains open until the client send a request to close the connection. Is a stateful communication model and server is aware of all the open connections.



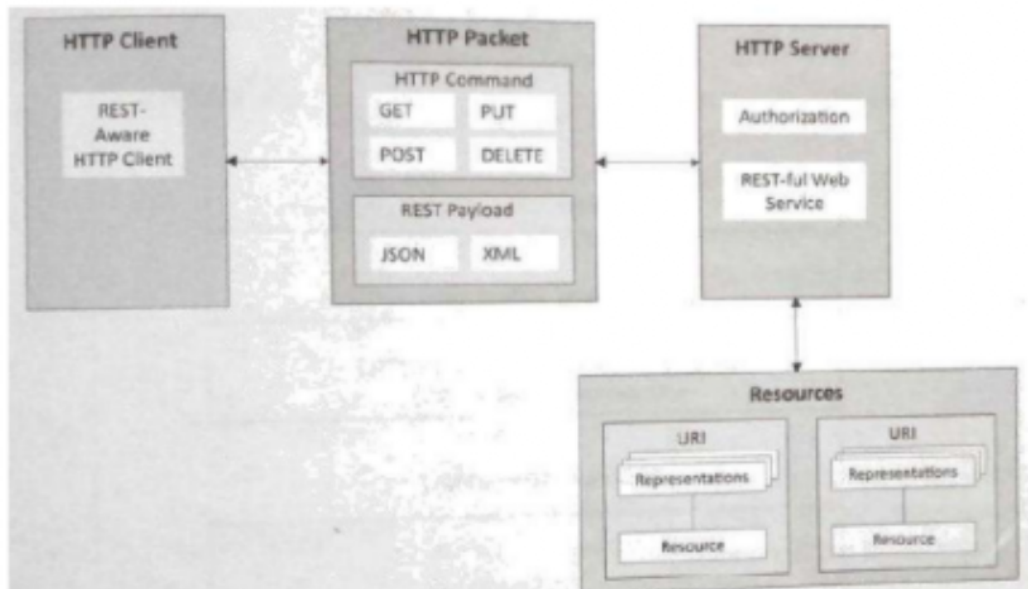
IoT Communication APIs:

- 1) REST based communication APIs (Request-Response Based Model).
- 2) WebSocket based Communication APIs (Exclusive Pair Based Model).

1) REST based communication APIs: Representational State Transfer (REST) is a set of architectural principles by which we can design web services

and web APIs that focus on a system 's resources and have resource states are addressed and transferred.

The REST architectural constraints: Fig. shows communication between client server with REST APIs.



Client-Server: The principle behind client-server constraint is the separation of concerns. Separation allows client and server to be independently developed and updated.

Stateless: Each request from client to server must contain all the info. Necessary to understand the request, and cannot take advantage of any stored context on the server.

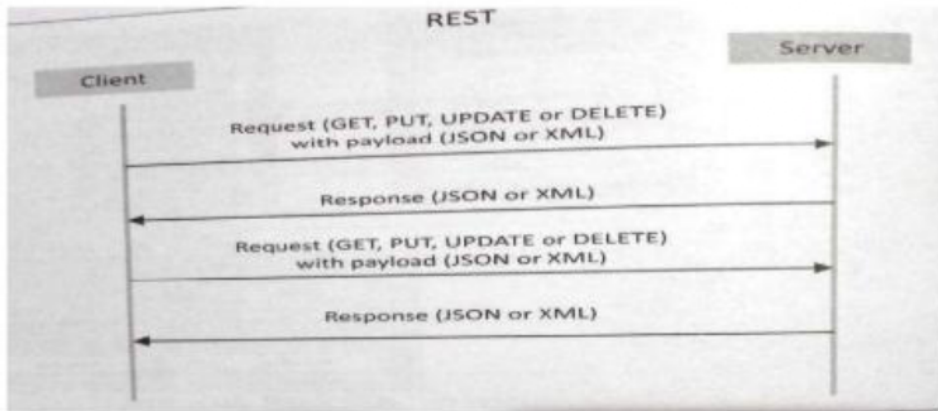
Cache-able: Cache constraint requires that the data within a response to a request be implicitly or explicitly labeled as cache-able or non-cacheable. If a response is cache-able, then a client cache is given the right to reuse that response data for later, equivalent requests.

Layered System: constraints the behavior of components such that each component cannot see beyond the immediate layer with which they are interacting.

User Interface: constraint requires that the method of communication between a client and a server must be uniform.

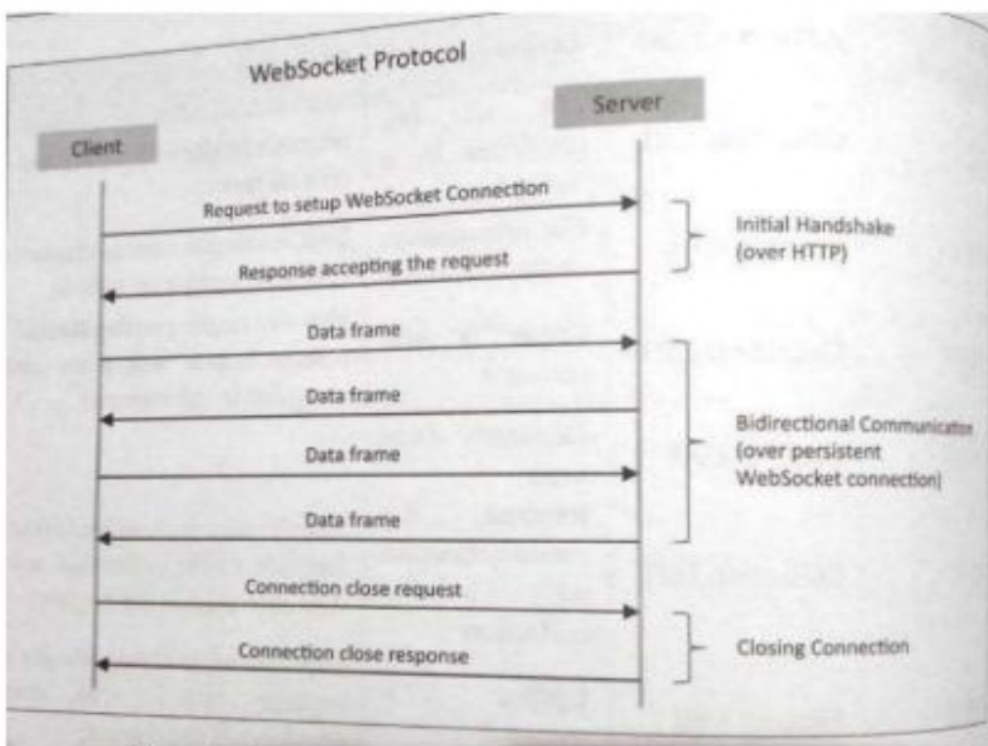
Code on Demand: Servers can provide executable code or scripts for clients to execute in their context. This constraint is the only one that is optional.

Request-Response model used by REST:



RESTful webservice is a collection of resources which are represented by URIs. RESTful web API has a base URI (e.g: <http://example.com/api/tasks/>). The clients send requests to these URIs using the methods defined by the HTTP protocol (e.g: GET, PUT, POST or DELETE). A RESTful web service can support various internet media types.

2) WebSocket Based Communication APIs: WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model.



9.Sensor Network

✔ What is a Sensor Network?

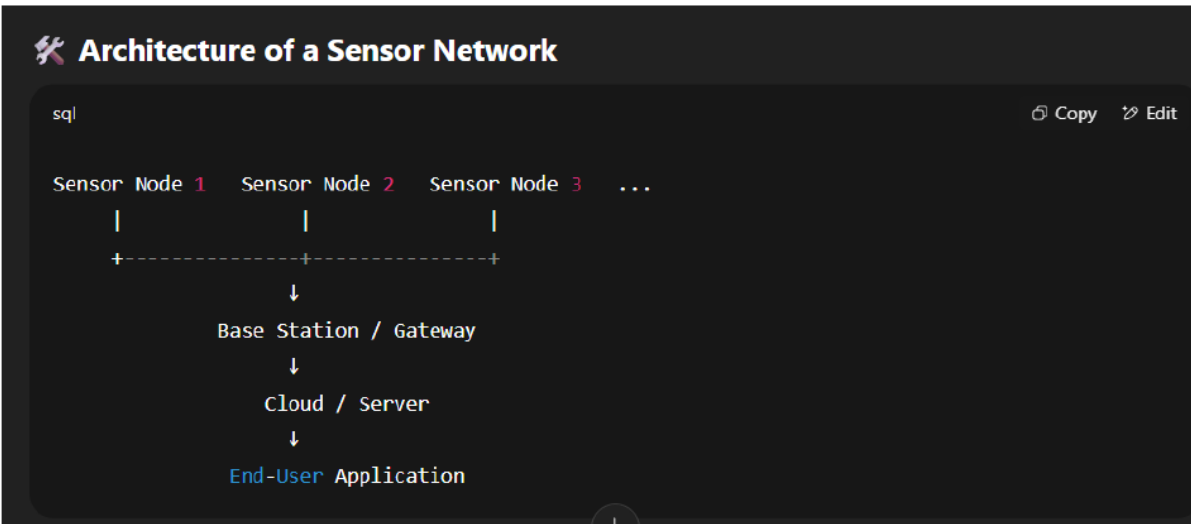
A **Sensor Network** is a group of spatially distributed and connected **sensor nodes** that **monitor physical or environmental conditions** (like temperature, humidity, pressure, etc.) and **communicate** the collected data to a central location (base station or cloud) for processing.

It is also known as a **Wireless Sensor Network (WSN)** when sensors communicate wirelessly.

🔧 Components of a Sensor Node

Each sensor node typically consists of:

Component	Function
Sensor	Senses physical/environmental data
Microcontroller	Processes the sensed data
Transceiver	Sends/receives data over the network
Battery	Provides power supply
Memory	Stores data and instructions





Features of Sensor Networks

Feature	Description
Scalability	Can support large number of nodes
Fault Tolerance	Network can adapt to node failures
Energy Efficiency	Low power usage is crucial for battery-operated nodes
Self-configuring	Nodes can join or leave without manual setup
Multi-hop Routing	Data can be passed through intermediate nodes



Types of Sensor Networks

Type	Description
Terrestrial WSN	Deployed on land (agriculture, smart cities)
Underground WSN	Installed under soil (soil monitoring)
Underwater WSN	Used in ocean/water monitoring
Multimedia WSN	Supports video/image/audio transmission
Mobile WSN	Nodes are mobile (e.g., in drones or robots)



Applications of Sensor Networks in IoT

Domain	Application
Smart Agriculture	Monitoring soil moisture, crop health
Smart Homes	Motion detection, fire/smoke sensors
Healthcare	Patient health monitoring (wearables)
Environmental	Air pollution, water quality, weather
Industrial IoT	Machine temperature, vibration detection

Challenges in Sensor Networks

- **Energy consumption** (limited battery life)
- **Network coverage and connectivity**
- **Data security and privacy**
- **Latency and real-time response**

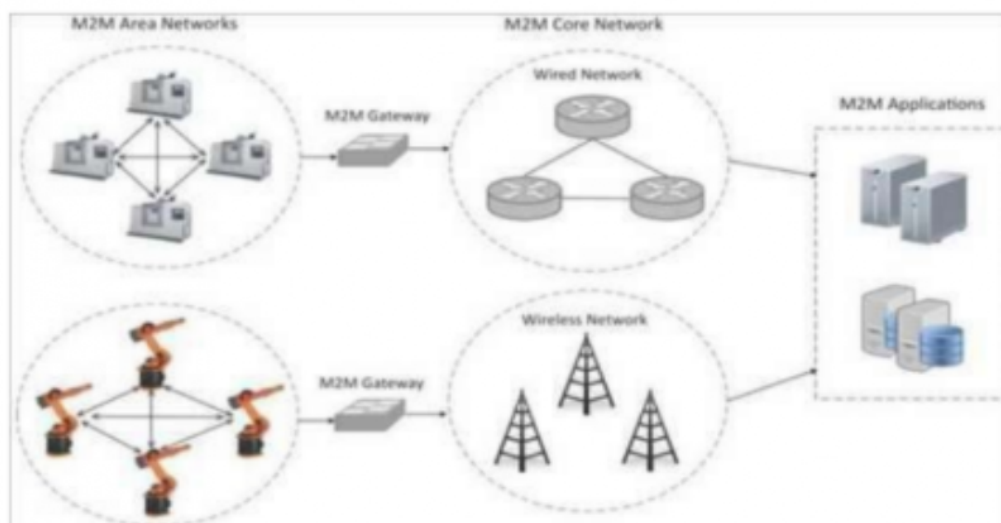
UNIT-2

1.M2M

MACHINE-TO-MACHINE (M2M)

COMMUNICATIONS

- ❏ Machine-to-Machine (M2M) refers to networking of machines (or devices) for the purpose of remote monitoring and control and data exchange.
- ❏ Term which is often synonymous with IoT is Machine-to-Machine (M2M).
- ❏ IoT and M2M are often used interchangeably.
- ❏ Fig. Shows the end-to-end architecture of M2M systems comprises of M2M area networks, communication networks and application foaming.



- ❏ An M2M area network comprises of machines (or M2M nodes) which have embedded network modules for sensing, actuation and communicating various communication protocols can be used for M2M LAN such as ZigBee, Bluetooth, M-bus, Wireless M-Bus etc., These protocols provide connectivity between M2M nodes within an M2M area network.
- ❏ The communication network provides connectivity to remote M2M area networks. The communication network provides connectivity to remote M2M area network. The communication network can use either wired or wireless network (IP based). While the M2M are networks use either

proprietary or non-IP based communication protocols, the communication network uses IP-based network. Since non-IP based protocols are used within M2M area network, the M2M nodes within one network cannot communicate with nodes in an external network.

❗ To enable the communication between remote M2M are network, M2M gateways are used.

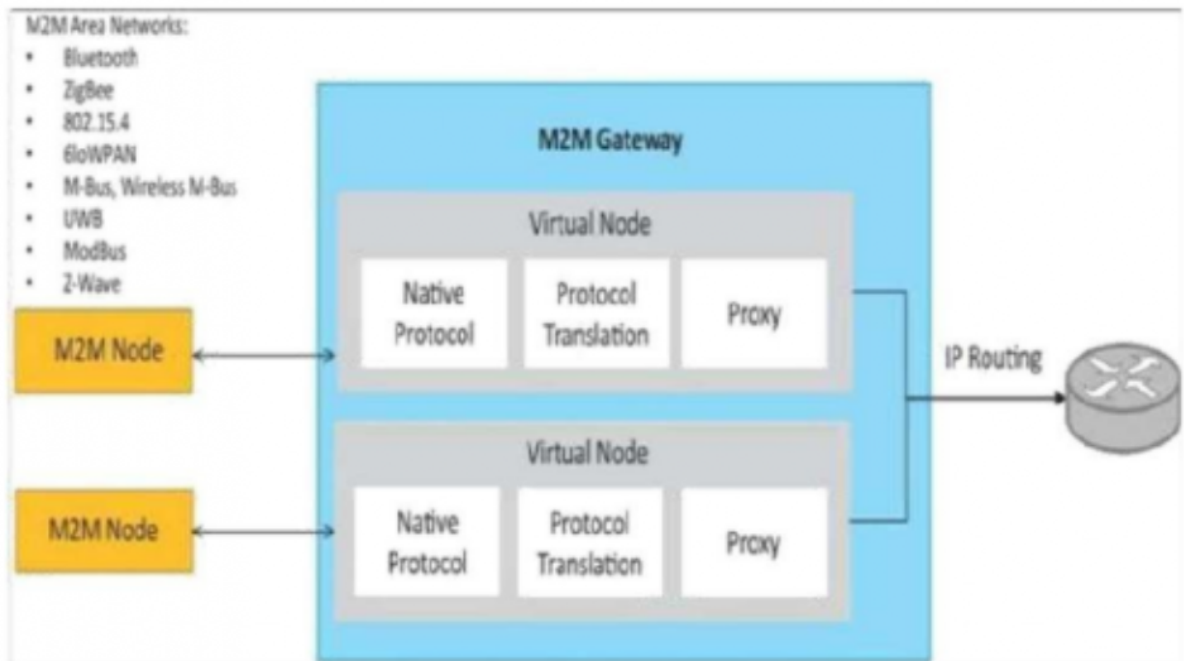


Fig. Shows a block diagram of an M2M gateway. The communication between M2M nodes and the M2M gateway is based on the communication protocols which are naive to the M2M are network. M2M gateway performs protocol translations to enable Ip-connectivity for M2M are networks. M2M gateway acts as a proxy performing translations from/to native protocols to/from Internet Protocol (IP). With an M2M gateway, each mode in an M2M area network appears as a virtualized node for external M2M area networks.

Difference Between IoT and M2M

Difference Between M2M and IoT

Aspect	M2M (Machine-to-Machine)	IoT (Internet of Things)
Definition	Direct communication between two machines or devices	A network of interconnected devices that collect, share, and act on data via the Internet
Scope	Narrow and focused (device-to-device communication)	Broad (includes people, processes, cloud, analytics, etc.)
Communication	Point-to-point or through a private network	Internet-based, often cloud-centric
Technology Used	GSM, SMS, Ethernet	Wi-Fi, Bluetooth, ZigBee, MQTT, CoAP, Cloud, AI/ML
Intelligence	Limited, rule-based automation	Smart, uses data analytics and decision-making
Interoperability	Generally vendor-specific and limited	Highly interoperable (multiple platforms/devices)
Data Access	Local (no cloud)	Remote access via cloud/web/mobile
Example	Smart meters sending data to a server	Smart home system adjusting lights and AC based on user presence

★ Key Differences in Simple Terms

- **M2M** is like two machines talking directly (e.g., a vending machine notifying stock status).
- **IoT** is a **smart network** of devices (e.g., your fridge ordering milk when it's low).

✓ Summary

- **M2M** is a **subset** of **IoT**.
- **M2M** is limited to **device-level communication**, whereas **IoT** involves **devices + cloud + apps + analytics**.
- **IoT** is **M2M + Internet + Intelligence**.

2. Interoperability in Iot

INTEROPERABILITY IN IOT

What does interoperability mean?

Defining Interoperability: Interoperability is the ability of different information technology systems and software applications to communicate, to exchange data accurately, effectively, and consistently, and to use the information that has been exchanged.

What is interoperability and why is it especially important in the Internet?

Foundational interoperability enables one information system to exchange data with another information system. The system receiving this information does not need to have to interpret the data. It will be instantly available for use.

Dimensions for Interoperability: The main objective of this report is not to produce a new definition on interoperability but to explore the different roles and functionality that interoperability plays in IoT. There are many definitions of interoperability; we try to provide a common definition that can be extracted from many of those definitions (bringing from the 3rd Generation Partnership Project, 3GPP). Interoperability is:

“the ability of two or more systems or components to exchange data and use information”

This definition provides many challenges on how to:

- 🔧 Get the information.
- 🔧 Exchange data, and
- 🔧 Use the information in understanding it and being able to process it.

A simple representation of interoperability can be seen as follow:

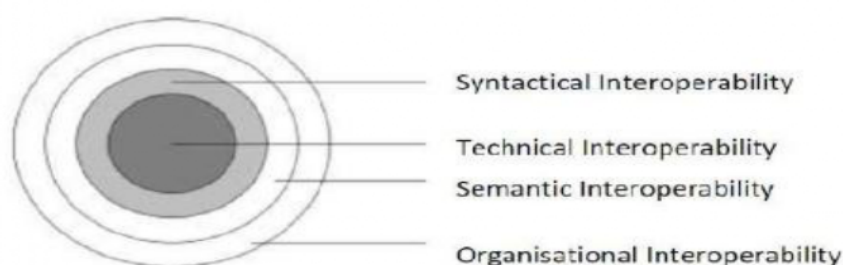


Figure 1 The Dimensions of Interoperability

Technical Interoperability is usually associated with hardware/software components, systems and platforms that enable machine-to-machine communication to take place. This kind of interoperability is often centred on (communication) protocols and the infrastructure needed for those protocols to operate.

Syntactical Interoperability is usually associated with data formats. Certainly, the messages transferred by communication protocols need to have a well-defined syntax and encoding, even if it is only in the form of bittables. However, many protocols carry data or content, and this can be represented using high-level syntaxes such as HTML or XML

Semantic Interoperability is usually associated with the meaning of content and concerns the human rather than machine interpretation of the content. Thus, interoperability on this level means that there is a common understanding between people of the meaning of the content (information) being exchanged.

Technical Interoperability is usually associated with hardware/software components, systems and platforms that enable machine-to-machine communication to take place. This kind of interoperability is often centred on (communication) protocols and the infrastructure needed for those protocols to operate.

Syntactical Interoperability is usually associated with data formats. Certainly, the messages transferred by communication protocols need to have a well-defined syntax and encoding, even if it is only in the form of bittables. However, many protocols carry data or content, and this can be represented using high-level syntaxes such as HTML or XML

Semantic Interoperability is usually associated with the meaning of content and concerns the human rather than machine interpretation of the content. Thus, interoperability on this level means that there is a common understanding between people of the meaning of the content (information) being exchanged.

Interoperability: Challenges and Requirements: The overall challenges in interoperability is first to stabilize the foundation of the real world data/services, ensuring technical interoperability from technologies to deliver mass of information and then complementary challenges are for the information to be understood and processed. Before entering into details

Interoperability in IoT

Definition

Interoperability in IoT refers to the **ability of different IoT devices, platforms, systems, and applications to communicate, exchange data, and work together seamlessly**, regardless of their manufacturer, protocol, or technology.

Why is Interoperability Important?

In a typical IoT ecosystem, devices from different vendors are used. Interoperability ensures that these devices can:

- Share data correctly
 - Understand each other's formats
 - Work together in a coordinated way
 - Prevent vendor lock-in
-

Types of Interoperability

Type	Description
Device Interoperability	Devices from different manufacturers can connect and exchange data
Network Interoperability	Different communication protocols (e.g., Wi-Fi, ZigBee, LoRa) can coexist
Data Interoperability	Data formats (JSON, XML) are compatible and understood by all systems
Application Interoperability	Different apps/services can use shared data and provide integrated functionality

Examples in Real Life

Scenario	Interoperability Feature
Smart home using Alexa, Google Home, and Philips Hue	Device + Application interoperability
Wearable fitness tracker syncing with mobile and cloud	Device + Network + Data interoperability

How is Interoperability Achieved in IoT?

- **Open Standards:** Use of common protocols (MQTT, CoAP, HTTP, JSON, XML)
 - **Middleware:** Acts as a bridge between incompatible devices/systems
 - **APIs:** Allow applications to access data from various sources
 - **Gateways:** Convert one protocol to another (e.g., ZigBee to Wi-Fi)
-

Challenges in Interoperability

Challenge	Description
Proprietary protocols	Vendors use closed systems that don't share data easily
Lack of standardization	Too many competing standards in IoT
Security and compatibility	Sharing data across systems may introduce risks

3.Introduction to Arduino Programmming

INTRODUCTION TO ARDUINO PROGRAMMING

Arduino Programs:

All Arduino programs must follow the following main structure:

```
// Initialization, define variables, etc.  
  
void setup ()  
{  
    // Initialization  
    ...  
}  
  
void loop ()  
{  
    //Main Program  
    ...  
}
```

Arduino Program – Example:

```
void setup()  
{  
    pinMode(11, OUTPUT);    //Set the Pin as an Output  
}  
  
void loop()  
{  
    digitalWrite(11, HIGH); // Turn on the LED  
    delay(1000);            // Wait for one second  
    digitalWrite(11, LOW);  // Turn off the LED  
    delay(1000);            // Wait for one second  
}
```

Arduino Program – Using Comments:

```
void setup()
{
    pinMode(11, OUTPUT);    //Set the Pin as an Output
}

void loop()
{
    digitalWrite(11, HIGH); // Turn on the LED

    /*
    ... This will not be executed by the program because
    it is a comment...
    */
}
```

Creating and Using Functions:

```
int z;

void setup()
{
}

void loop()
{
    z = calculate(2,3);
}

float calculate(int x, int y)
{
    return (x + y);
}
```

Using the Function

Creating the Function

TRY IT OUT!

Here are some Arduino Examples you should try.

Make sure your Arduino is connected to the PC and start the Code Editor



” Hello World” Example

TRY IT OUT!

Create the following program:

Open the” Serial Monitor” in order to see the output

```
void setup()  
{  
  Serial.begin(9600);  
  Serial.println("Hello, world!");  
}  
  
void loop()  
{  
}
```

"Hello World" Example

TRY IT OUT!

Create the following program:

Open the "Serial Monitor" in order to see the output

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  Serial.println("Hello,
world!"); delay(1000);
}
```

Example

Create the following program:

Open the "Serial Monitor" in order to see the output

```
int z;int a;int b;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  a = random(100);
  b = random(100);
  z = calculate(a,b); //Adding 2 Numbers

  //Write Values to Serial Monitor
  Serial.print(a);
  Serial.print(" + ");
  Serial.print(b);
  Serial.print(" = ");
  Serial.println(z);

  delay(1000);
}
float calculate(int x, int y)
{
  return (x + y);
}
```

TRY IT OUT!

Creating Functions

TRY IT OUT!

Create a function that calculates the area of a circle with a given radius.

Write the result to the Serial Monitor.



Solution

```
void setup()
{
    Serial.begin(9600);
    // calculate the area of a circle with radius of 9.2
    float r=9.2;
    area = CircleArea(r);
    Serial.print("Area of circle is: ");
    // print area to 4 decimal places
    Serial.println(area, 4);
}

void loop()
{
}

// calculate the area of a circle
float CircleArea(float radius)
{
    float result;
    const float pi = 3.14;

    result = pi * radius * radius;

    return result;
}
```

TRY IT OUT!

For Loop

In this program we use a For Loop to find the Sum of 100 Random Numbers.

Then we find the Average.

The Sum and Average should be written to the Serial Monitor.

```
int x; int sum = 0; float gjennomsnitt = 0;
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  sum = 0;
  for (int i = 0; i<100; i++)
  {
    x = random(100);
    sum = sum + x;
  }

  average = sum / 100;
  Serial.print(" Sum = ");
  Serial.print(sum);
  Serial.print(" ,
  Average = ");
  Serial.println(average);
  delay(1000);
}
```

TRY IT OUT!

Arrays

Here we shall use arrays in the Arduino program

Create this program from scratch and open the Serial Monitor to see the result.

```
const int arraysize = 100;
int x;
int sum = 0;

int myarray[arraysize];
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  sum = 0;
  for (int i = 0; i < arraysize; i++)
  {
    x = random(100);
    myarray[i] = x;
  }
  sum = calculateSum(myarray);
  average = sum / 100;
  Serial.print(" Sum = ");
  Serial.print(sum);
  Serial.print(" , Average = ");
  Serial.println(average);
  delay(1000);
}

int calculateSum (int myarray[])
{
  for (int i = 0; i < arraysize; i++)
  {
    sum = sum + myarray[i];
  }
  return sum;
}
```

TRY IT OUT!

4.Integration of Sensor and Actuator with Arduino

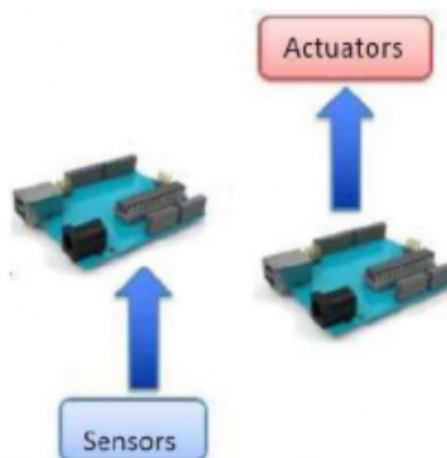
INTEGRATION OF SENSORS AND ACTUATORS WITH ARDUINO

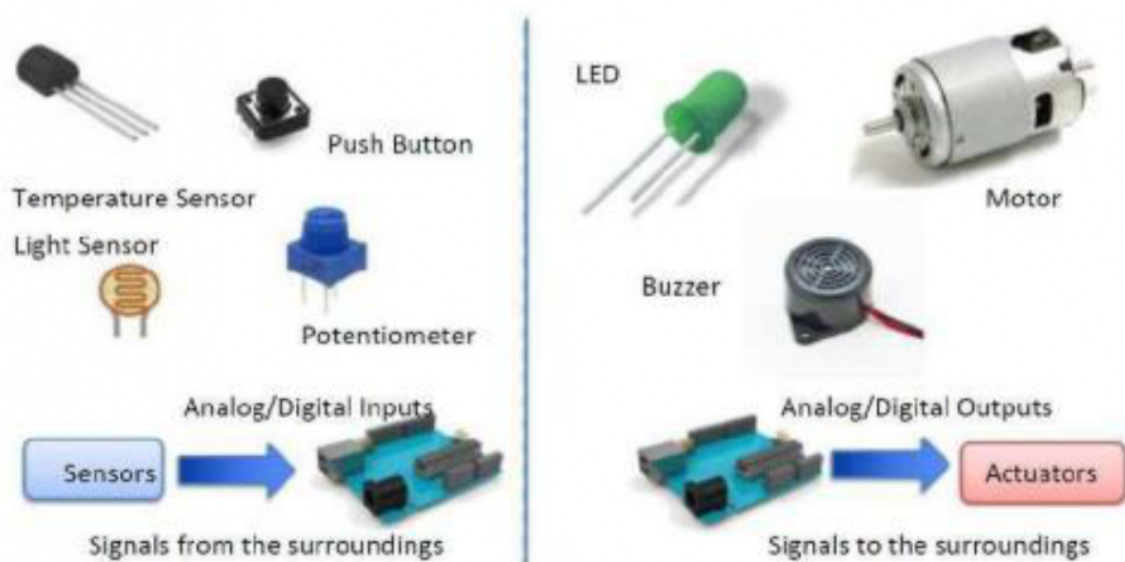
Sensors and Actuators:

- ✎ A **Sensor** is a converter that measures a physical size and converts it to a signal that can be read by an instrument, data acquisition device, or an Arduino. **Examples:** temperature sensor, pressure sensor, etc.
- ✎ An **Actuator** is a kind of motor that moves or controls a mechanism or system. It is powered by an energy source, typical electric current, hydraulic fluid pressure, or air pressure, and converts this energy into motion. **Examples:** Engine, Pump, Valve, etc.
- ✎ The sensors and actuators can be either digital or analog.
- ✎ Some sensors and actuators have been made for Arduino, while others need to be connected in some circuit to work properly with Arduino.
- ✎ Many of these come with ready-made libraries for Arduino, so they are easy to use.

Examples:

- 1) Electrical Circuit.
- 2) Blinking LED.
- 3) Switch.
- 4) Potentiometer.
- 5) Temperature.
- 6) Light Sensor.
- 7) Thermistor.





Programming

Program Structure

```
//Global variable
...

void setup()
{
  //Initialization
}

{
  //Main Program
}
```

Example 2

You need to use the following:

Which Pin (3, ...) are you using?

pinMode

digitalWrite(pin, value);

A Digital Pin can have 2 values, either

delay(ms);

The delay() function make a small pause

In milliseconds (ms), e.g., delay(1000)

pause the program for 1 second

A Digital Pin can either be an INPUT or an OUTPUT. Since we shall use it to turn-on a LED, we set it to OUTPUT.

Turn-on LED Turn-off LED

or LOW

Arduino Program

Example 2

```
void setup()
{
  pinMode(8, OUTPUT);
}

void loop()
{
  digitalWrite(8,      // Turn on the LED
    HIGH); delay(1000); // Wait for one second
  digitalWrite(8, LOW); // Turn off the LED
  delay(1000);          // Wait for one second
}
```

TRY IT OUT!
